DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

COURSE WORK & PRACTICAL LABORATORY RECORD

OFFICIAL LABORATORY RECORD

Data Structures & Algorithms Laboratory

Practical Implementation of Core DSA Concepts in Standard C

Total Experiments: **21** • Language: **C (GCC / C99)** • Coverage: **Code, Sample Input & Terminal Output**

STUDENT NAME	**SIDHARTH**
ROLL NUMBER	**40**
COURSE / LAB	**Data Structures Laboratory (DSA)**
SEMESTER / BRANCH	**Semester 3 / B.Tech CSE**
RECORD STATUS	**Completed & Verified**

CERTIFICATE: Certified that this is the bona fide record of practical work completed by **Sidharth (Roll No: 40)** in the Data Structures Laboratory during the academic year 2026–2027.

Student Signature | Staff-in-Charge | Head of Department

Comprehensive Laboratory Compilation • Academic Year 2026–2027

INDEX OF EXPERIMENTS

Data Structures Laboratory Record • Sidharth (Roll No: 40)

21 Experiments Total

NO.	EXPERIMENT TITLE & FILE	OBJECTIVE / AIM	PAGE	STATUS
01	**Matrix Addition** `1_matrix_add.c`	To write a C program to perform addition of two matrices of order m × n.	4	Verified
02	**Polynomial Addition using Structure** `2_polynomial_add.c`	To write a C program to add two polynomials using an array of structures.	5	Verified
03	**Sparse Matrix Representation (Tuple Form)** `3_matrix_to_sparse.c`	To write a C program to convert a given standard matrix into its 3-tuple (sparse matrix) representation.	7	Verified
04	**Sparse Matrix Addition** `4_sparse_matrix_add.c`	To write a C program to perform addition of two sparse matrices represented in triplet form.	9	Verified
05	**Transpose of a Sparse Matrix** `5_sparse_trans.c`	To write a C program to compute and display the transpose of a sparse matrix in triplet form.	11	Verified
06	**Stack Implementation using Array** `6_stack.c`	To implement basic stack operations (Push, Pop, Display) using an array in C.	13	Verified
07	**Linear Queue Implementation using Array** `7_standard_queue.c`	To implement linear queue operations (Enqueue, Dequeue, Display) using an array in C.	16	Verified
08	**Circular Queue Implementation using Array** `8_circular_queue.c`	To implement circular queue operations (Enqueue, Dequeue, Display) with circular wraparound modulo indexing.	19	Verified
09	**Double Ended Queue (Deque) Implementation** `9_dequeue.c`	To implement a double-ended queue supporting insertion and deletion operations at both front and rear ends.	22	Verified
10	**Infix to Postfix Conversion** `10_infixtopostfix.c`	To write a C program to convert an infix mathematical expression to postfix notation using a stack.	26	Verified
11	**Postfix Expression Evaluation** `11_postfixeval.c`	To write a C program to evaluate a postfix mathematical expression using stack operands.	28	Verified
12	**Singly Linked List Implementation** `12_single_linked_list.c`	To implement singly linked list operations including insertion and deletion at beginning, middle, and end, with traversal.	30	Verified
13	**Doubly Linked List Implementation** `13_doubly_linked_list.c`	To implement doubly linked list operations including bidirectional links, insertion and deletion at positions, and traversal.	34	Verified
14	**Stack Implementation using Linked List** `14_stack_linked_list.c`	To implement dynamic stack data structure operations (Push, Pop, Display) using a singly linked list.	38	Verified
15	**Linear Search Algorithm** `15.c`	To search for a given target element in an array using the linear (sequential) search algorithm.	41	Verified
16	**Binary Search Algorithm** `16.c`	To search for a target value in a sorted array using binary search with middle element comparisons.	42	Verified
17	**Insertion Sort Algorithm** `17.c`	To sort an array of integers in ascending order using insertion sort with step-by-step trace.	44	Verified
18	**Selection Sort Algorithm** `18.c`	To sort an array of integers in ascending order using selection sort with step-by-step trace.	46	Verified
19	**Quick Sort Algorithm** `19.c`	To sort an array of integers using quick sort algorithm based on partitioning and divide-and-conquer strategy.	48	Verified
20	**Merge Sort Algorithm** `20.c`	To sort an array of integers using merge sort algorithm based on recursive sub-array division and merging.	51	Verified

NO.	EXPERIMENT TITLE & FILE	OBJECTIVE / AIM	PAGE	STATUS
21	**Binary Tree Traversals** `21.c`	To construct a binary tree dynamically and implement Depth-First (Pre-order, In-order, Post-order) and Breadth-First (Level-order) traversals.	53	Verified

EXPERIMENT 01 `1_matrix_add.c`

Matrix Addition

AIM & OBJECTIVE

To write a C program to perform addition of two matrices of order m × n.

ALGORITHM / LOGIC HIGHLIGHTS

- Input the row and column dimensions of the matrices.
- Input the elements of the first matrix A and second matrix B.
- Compute each element of the sum matrix as C[i][j] = A[i][j] + B[i][j].
- Display the resultant sum matrix in tabular matrix format.

SOURCE CODE (C IMPLEMENTATION)

1_matrix_add.c — C (C99 STANDARD)

```c
/* Matrix Addition */

#include <stdio.h>

int main() {
    int row, col;

    printf("Enter row & column sizes of 1st matrix: ");
    scanf("%d%d", &row, &col);
    int A[row][col], B[row][col];

    for (int count = 1; count < 3; count++) {
        const char *matrix_number = (count == 1) ? "1st" : "2nd";
        int (*matrix)[col] = (count == 1) ? A : B;
        printf("Enter %d elemnts for %s matrix: ", row*col, matrix_number);

        for (int i=0; i < row; i++)
            for (int j=0; j < col; j++)
                scanf("%d", &matrix[i][j]);
    }

    printf("Added:\n");
    for (int i=0; i < row; i++) {
        for (int j=0; j < col; j++)
            printf("%d ", A[i][j] + B[i][j]);
        printf("\n");
    }

    return 0;
}
```

INPUT PARAMETERS & STREAM

Matrix Dimensions: 2 × 2
Matrix 1: [[1, 2], [3, 4]]
Matrix 2: [[5, 6], [7, 8]]

Terminal Feed / Stdin: `2 2 | 1 2 3 4 | 5 6 7 8`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

terminal - sidharth@lab-pc: ~/dsa/exp1

```
$ ./1_matrix_add
Enter row & column sizes of 1st matrix: 2 2
Enter 4 elemnts for 1st matrix: 1 2 3 4
Enter 4 elemnts for 2nd matrix: 5 6 7 8
Added:
6 8
10 12

Process finished with exit code 0
```

EXPERIMENT 02 `2_polynomial_add.c`

Polynomial Addition using Structure

AIM & OBJECTIVE

To write a C program to add two polynomials using an array of structures.

ALGORITHM / LOGIC HIGHLIGHTS

- Define a structure Poly with fields coeff (coefficient) and exp (exponent).
- Read number of terms and term pairs (coeff, exp) for both polynomials in descending order of exponents.
- Traverse both polynomials simultaneously using pointers i and j. If exponents are equal, add coefficients. Otherwise, insert the term with higher exponent into the result polynomial.
- Append remaining terms from either polynomial and display the resulting sum expression.

SOURCE CODE (C IMPLEMENTATION)

2_polynomial_add.c — C (C99 STANDARD)

```c
// Addition of 2 polynomials using structure

#include <stdio.h>

struct Poly {
    int coeff, exp;
};


int main() {
    int size_1, size_2;

    printf("Enter number of elements for 1st Polynomial: ");
    scanf("%d", &size_1);
    printf("Enter number of elements for 2nd Polynomial: ");
    scanf("%d", &size_2);

    struct Poly A[size_1], B[size_2], SUM[size_1+size_2];

    for (int count = 1; count < 3; count++) {
        const char *polynomial_number = (count == 1) ? "1st" : "2nd";
        struct Poly *polynomial = (count == 1) ? A : B;
        int size = (count == 1) ? size_1 : size_2;
        printf("\nEnter %d elements for %s polynomial:\n", size, polynomial_number);

        for (int i=0; i < size; i++)
            scanf("%d%d", &polynomial[i].coeff, &polynomial[i].exp);
    }

    int i = 0, j = 0, k = 0;
    while (i < size_1 && j < size_2) {
        if (A[i].exp == B[j].exp) {
            SUM[k].coeff = A[i].coeff + B[j].coeff;
            SUM[k].exp = A[i].exp;
            i++; j++;
        } else if (A[i].exp > B[j].exp) {
            SUM[k] = A[i];
            i++;
        } else {
            SUM[k] = B[j];
            j++;
        }
        k++;
    }

    for (i; i < size_1; i++, k++) SUM[k] = A[i];
    for (j; j < size_2; j++, k++) SUM[k] = B[j];

    printf("Sum:\n");
    for (i = 0; i < k; i++) {
        printf("(%d**%d)", SUM[i].coeff, SUM[i].exp);
        if (i < k-1) printf(" + ");
    }

    return 0;
}
```

INPUT PARAMETERS & STREAM

Polynomial 1 (3 terms): $4x^3 + 3x^2 + 5x^0$
Polynomial 2 (3 terms): $2x^3 + 1x^1 + 2x^0$

Terminal Feed / Stdin: `3 | 3 | 4 3 | 3 2 | 5 0 | 2 3 | 1 1 | 2 0`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp2

$ ./2_polynomial_add
Enter number of elements for 1st Polynomial: 3
Enter number of elements for 2nd Polynomial: 3

Enter 3 elements for 1st polynomial:
4 3
3 2
5 0

Enter 3 elements for 2nd polynomial:
2 3
1 1
2 0
Sum:
(6**3) + (3**2) + (1**1) + (7**0)

Process finished with exit code 0
```

EXPERIMENT 03 `3_matrix_to_sparse.c`

Sparse Matrix Representation (Tuple Form)

AIM & OBJECTIVE

To write a C program to convert a given standard matrix into its 3-tuple (sparse matrix) representation.

ALGORITHM / LOGIC HIGHLIGHTS

- Input the row and column dimensions and read all matrix elements.
- Initialize sparse representation with header row: sparse[0][0] = row, sparse[0][1] = col.
- Scan the matrix row by row. For every non-zero element at (i, j), record row i, column j, and value in the triplet array.
- Store total non-zero count in sparse[0][2] and print the resulting 3-tuple rows.

SOURCE CODE (C IMPLEMENTATION)

3_matrix_to_sparse.c — C (C99 STANDARD)

```c
/* Tuple Representation of Sparse Matrix */

#include <stdio.h>

int main() {
    int row, col, number_non_zero = 0;
    printf("Enter row & column size: ");
    scanf("%d%d", &row, &col);

    int matrix[row][col];
    printf("Enter %d elements: ", row*col);
    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++) {
            scanf("%d", &matrix[i][j]);
    }

    int sparse[number_non_zero+1][3], r = 1;
    sparse[0][0] = row;
    sparse[0][1] = col;

    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++)
            if (matrix[i][j] != 0) {
                sparse[r][0] = i;
                sparse[r][1] = j;
                sparse[r][2] = matrix[i][j];
                r++;
                number_non_zero++;
            }

    sparse[0][2] = number_non_zero;

    for (int i = 0; i <= number_non_zero; i++) {
        printf("(");
        for (int j = 0; j < 3; j++) {
            printf("%d", sparse[i][j]);
            if (j < 2) printf(", ");
        }
        printf(")\n");
    }

    return 0;
}
```

INPUT PARAMETERS & STREAM

Matrix Size: 3 × 3
Elements: [[1, 0, 0], [0, 0, 2], [0, 3, 0]]

Terminal Feed / Stdin: `3 3 | 1 0 0 0 0 2 0 3 0`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp3

$ ./3_matrix_to_sparse
Enter row & column size: 3 3
```

```
Enter 9 elements: 1 0 0 0 0 2 0 3 0
(3, 3, 3)
(0, 0, 1)
(1, 2, 2)
(2, 1, 3)

Process finished with exit code 0
```

EXPERIMENT 04 `4_sparse_matrix_add.c`

Sparse Matrix Addition

AIM & OBJECTIVE

To write a C program to perform addition of two sparse matrices represented in triplet form.

ALGORITHM / LOGIC HIGHLIGHTS

- Read dimensions and non-zero triplets (row, col, value) for Matrix 1 and Matrix 2.
- Initialize result matrix header with same dimensions.
- Use two-pointer traversal: compare row indices first, then column indices. If positions match, add values. If one index is strictly smaller, copy it to the result.
- Append remaining non-zero entries from either matrix and display resultant triplet matrix.

SOURCE CODE (C IMPLEMENTATION)

4_sparse_matrix_add.c — C (C99 STANDARD)

```c
/* Sparse Matrix Addition */

#include <stdio.h>

int main() {
    int rows, cols, non_zero1, non_zero2;
    printf("Enter total number of rows and columns for the matrices: ");
    scanf("%d%d", &rows, &cols);
    printf("Enter number of non-zero elements for Matrix 1: ");
    scanf("%d", &non_zero1);
    printf("Enter number of non-zero elements for Matrix 2: ");
    scanf("%d", &non_zero2);

    int sparse1[non_zero1 + 1][3];
    int sparse2[non_zero2 + 1][3];
    int sum[non_zero1 + non_zero2 + 1][3];

    sparse1[0][0] = rows; sparse1[0][1] = cols; sparse1[0][2] = non_zero1;
    sparse2[0][0] = rows; sparse2[0][1] = cols; sparse2[0][2] = non_zero2;
    sum[0][0] = rows; sum[0][1] = cols;

    printf("\nEnter Triplets for Matrix 1 (Row, Column, Value):\n");
    for (int i = 1; i <= non_zero1; i++) {
        scanf("%d%d%d", &sparse1[i][0], &sparse1[i][1], &sparse1[i][2]);
    }

    printf("\nEnter Triplets for Matrix 2 (Row, Column, Value):\n");
    for (int i = 1; i <= non_zero2; i++) {
        scanf("%d%d%d", &sparse2[i][0], &sparse2[i][1], &sparse2[i][2]);
    }

    int i = 1, j = 1, k = 1;

    while (i <= non_zero1 && j <= non_zero2) {
        if (sparse1[i][0] == sparse2[j][0]) {
            if (sparse1[i][1] == sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2] + sparse2[j][2];
                i++; j++; k++;
            }
            else if (sparse1[i][1] < sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2];
                i++; k++;
            }
            else {
                sum[k][0] = sparse2[j][0];
                sum[k][1] = sparse2[j][1];
                sum[k][2] = sparse2[j][2];
                j++; k++;
            }
        }
        else if (sparse1[i][0] < sparse2[j][0]) {
            sum[k][0] = sparse1[i][0];
            sum[k][1] = sparse1[i][1];
            sum[k][2] = sparse1[i][2];
```

```
            i++; k++;
        }
        else {
            sum[k][0] = sparse2[j][0];
            sum[k][1] = sparse2[j][1];
            sum[k][2] = sparse2[j][2];
            j++; k++;
        }
    }

    while (i <= non_zero1) {
        sum[k][0] = sparse1[i][0];
        sum[k][1] = sparse1[i][1];
        sum[k][2] = sparse1[i][2];
        i++; k++;
    }

    while (j <= non_zero2) {
        sum[k][0] = sparse2[j][0];
        sum[k][1] = sparse2[j][1];
        sum[k][2] = sparse2[j][2];
        j++; k++;
    }

    sum[0][2] = k - 1;

    printf("\nResultant Sparse Matrix (Triplet Representation):\n");
    printf("Row\tCol\tValue\n");
    for (int m = 0; m < k; m++) {
        printf("%d\t%d\t%d\n", sum[m][0], sum[m][1], sum[m][2]);
    }

    return 0;
}
```

INPUT PARAMETERS & STREAM

Matrix Size: 3 × 3
Matrix 1: 2 non-zeros [(0,0,5), (1,2,8)]
Matrix 2: 2 non-zeros [(0,0,3), (2,1,4)]

Terminal Feed / Stdin: `3 3 | 2 | 2 | 0 0 5 | 1 2 8 | 0 0 3 | 2 1 4`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp4

$ ./4_sparse_matrix_add
Enter total number of rows and columns for the matrices: 3 3
Enter number of non-zero elements for Matrix 1: 2
Enter number of non-zero elements for Matrix 2: 2

Enter Triplets for Matrix 1 (Row, Column, Value):
0 0 5
1 2 8

Enter Triplets for Matrix 2 (Row, Column, Value):
0 0 3
2 1 4

Resultant Sparse Matrix (Triplet Representation):
Row     Col     Value
3       3       3
0       0       8
1       2       8
2       1       4

Process finished with exit code 0
```

EXPERIMENT 05

5_sparse_trans.c

Transpose of a Sparse Matrix

AIM & OBJECTIVE

To write a C program to compute and display the transpose of a sparse matrix in triplet form.

ALGORITHM / LOGIC HIGHLIGHTS

- Read sparse matrix triplets (row, col, value) and initialize transpose header with swapped row/col bounds.
- For each non-zero element, swap row and column values.
- Sort the transposed triplets in lexicographical order (ascending row, then ascending col).
- Print the transposed triplet representation.

SOURCE CODE (C IMPLEMENTATION)

5_sparse_trans.c — C (C99 STANDARD)

```c
/* transpose of sparse matrix */

#include <stdio.h>

int main() {
    int rows, cols, non_zero;
    printf("Enter total number of rows and columns for the matrix: ");
    scanf("%d%d", &rows, &cols);
    printf("Enter number of non-zero elements for the matrix: ");
    scanf("%d", &non_zero);

    int sparse[non_zero + 1][3];
    sparse[0][0] = rows; sparse[0][1] = cols; sparse[0][2] = non_zero;

    printf("\nEnter Triplets for the Matrix (Row, Column, Value):\n");
    for (int i = 1; i <= non_zero; i++) {
        scanf("%d%d%d", &sparse[i][0], &sparse[i][1], &sparse[i][2]);
    }

    int transpose[non_zero + 1][3];
    transpose[0][0] = cols; transpose[0][1] = rows; transpose[0][2] = non_zero;

    // Perform transpose
    for (int i = 1; i <= non_zero; i++) {
        transpose[i][0] = sparse[i][1]; // Swap row and column
        transpose[i][1] = sparse[i][0];
        transpose[i][2] = sparse[i][2];
    }

    // Sort the transposed matrix based on row and column
    for (int i = 1; i <= non_zero; i++) {
        for (int j = i + 1; j <= non_zero; j++) {
            if ((transpose[i][0] > transpose[j][0]) ||
                (transpose[i][0] == transpose[j][0] && transpose[i][1] > transpose[j][1])) {

                for (int k = 0; k < 3; k++) {
                    int temp = transpose[i][k];
                    transpose[i][k] = transpose[j][k];
                    transpose[j][k] = temp;
                }
            }
        }
    }

    // Display the transposed matrix
    printf("\nThe transposed matrix is:\n");
    for (int i = 0; i <= non_zero; i++) {
        printf("%d %d %d\n", transpose[i][0], transpose[i][1], transpose[i][2]);
    }

    return 0;
}
```

INPUT PARAMETERS & STREAM

Matrix Size: 3 × 3
Non-zero count: 3
Triplets: (0,1,5), (1,2,8), (2,0,3)

Terminal Feed / Stdin: `3 3 | 3 | 0 1 5 | 1 2 8 | 2 0 3`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp5

$ ./5_sparse_trans
Enter total number of rows and columns for the matrix: 3 3
Enter number of non-zero elements for the matrix: 3

Enter Triplets for the Matrix (Row, Column, Value):
0 1 5
1 2 8
2 0 3

The transposed matrix is:
3 3 3
0 2 3
1 0 5
2 1 8

Process finished with exit code 0
```

EXPERIMENT 06 6_stack.c

Stack Implementation using Array

AIM & OBJECTIVE

To implement basic stack operations (Push, Pop, Display) using an array in C.

ALGORITHM / LOGIC HIGHLIGHTS

- Initialize stack top = -1 and define maximum capacity MAX = 10.
- Push(val): Check overflow (top >= MAX-1). If not full, increment top and insert val.
- Pop(): Check underflow (top == -1). If not empty, return items[top] and decrement top.
- Display(): Traverse and print items from index 0 to top.

SOURCE CODE (C IMPLEMENTATION)

6_stack.c C (C99 STANDARD)

```c
/* Implementation of a stack in C */

#include <stdio.h>
#define MAX 10

struct Stack {
    int top;
    int items[MAX];
};

void initialize(struct Stack *s) {
    s->top = -1;
}

void push(struct Stack *s, int val) {
    if (s->top >= MAX-1) {
        printf("Stack Overflow\n");
        return;
    }
    s->items[++(s->top)] = val;
}

int pop(struct Stack *s) {
    if (s->top == -1) {
        printf("Stack Underflow\n");
        return -1;
    }
    return s->items[s->top--];
}

void display(struct Stack s) {
    if (s.top == -1) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    for (int i=0; i<=s.top; i++) {
        printf("%d\n", s.items[i]);
    }
}

int main() {
    struct Stack s;
    initialize(&s);

    int choice, value;

    do {
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
```

```
                push(&s, value);
                break;
            case 2:
                value = pop(&s);
                if (value != -1) {
                    printf("Popped value: %d\n", value);
                }
                break;
            case 3:
                display(s);
                break;
            case 4:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}
```

INPUT PARAMETERS & STREAM

Operations: Push 10, Push 20, Push 30, Display, Pop (removes 30), Display, Exit

Terminal Feed / Stdin: `1 10 | 1 20 | 1 30 | 3 | 2 | 3 | 4`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

terminal - sidharth@lab-pc: ~/dsa/exp6

```
$ ./6_stack
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 30
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
20
30
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped value: 30
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
20
1. Push
2. Pop
3. Display
```

```
4. Exit
Enter your choice: 4
Exiting...

Process finished with exit code 0
```

EXPERIMENT 07 `7_standard_queue.c`

Linear Queue Implementation using Array

AIM & OBJECTIVE

To implement linear queue operations (Enqueue, Dequeue, Display) using an array in C.

ALGORITHM / LOGIC HIGHLIGHTS

- Initialize front = -1 and rear = -1 with maximum size MAX = 3.
- Enqueue(val): Check overflow (rear >= MAX-1). If empty, set front = 0. Increment rear and store val.
- Dequeue(): Check underflow (front == -1 or front > rear). Return items[front] and increment front.
- Display(): Iterate and print elements from front to rear.

SOURCE CODE (C IMPLEMENTATION)

7_standard_queue.c — C (C99 STANDARD)

```c
/* Implemetation of Standard Queue*/

#include <stdio.h>
#define MAX 3

struct Queue
{
    int front;
    int rear;
    int items[MAX];
};

void initialize(struct Queue *q) {
    q->front = -1;
    q->rear = -1;
}

void enqueu(struct Queue *q, int val) {
    if (q->rear >= MAX-1) {
        printf("Queue Overflow\n");
        return;
    }
    if (q->front == -1) q->front++;
    q->items[++(q->rear)] = val;
}

int dequeu(struct Queue *q) {
    if (q->front == -1 || q->front > q->rear) {
        printf("Queue Underflow\n");
        return -1;
    }
    return q->items[q->front++];
}

void display(struct Queue q) {
    if (q.front == -1 || q.front > q.rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("Displaying Contents:\n");
    for (int i=q.front; i<=q.rear; i++) {
        printf("%d\n", q.items[i]);
    }
}

int main() {
    struct Queue q;
    initialize(&q);

    int choice, value;

    do {
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

```

```c
        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueu(&q, value);
                break;
            case 2:
                value = dequeu(&q);
                if (value != -1) {
                    printf("Dequeued value: %d\n", value);
                }
                break;
            case 3:
                display(q);
                break;
            case 4:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}
```

INPUT PARAMETERS & STREAM

Operations: Enqueue 10, Enqueue 20, Enqueue 30, Display, Dequeue (removes 10), Display, Exit

Terminal Feed / Stdin: `1 10 | 1 20 | 1 30 | 3 | 2 | 3 | 4`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp7

$ ./7_standard_queue
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 20
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
10
20
30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 10
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
```

```
20
30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting...

Process finished with exit code 0
```

EXPERIMENT 08 `8_circular_queue.c`

Circular Queue Implementation using Array

AIM & OBJECTIVE

To implement circular queue operations (Enqueue, Dequeue, Display) with circular wraparound modulo indexing.

ALGORITHM / LOGIC HIGHLIGHTS

- Initialize front = 0, rear = MAX - 1, and count = 0.
- Enqueue(val): If count >= MAX, report overflow. Otherwise set rear = (rear + 1) % MAX, store val, and increment count.
- Dequeue(): If count == 0, report underflow. Retrieve val from front, set front = (front + 1) % MAX, and decrement count.
- Display(): Loop count times printing items[(front + i) % MAX].

SOURCE CODE (C IMPLEMENTATION)

8_circular_queue.c — C (C99 STANDARD)

```c
/* Circular Queue Implementation */

#include <stdio.h>
#define MAX 3

struct CircularQueue
{
    int front;
    int rear;
    int count;
    int items[MAX];
};

void initialize(struct CircularQueue *q) {
    q->front = 0;
    q->rear = MAX - 1;
    q->count = 0;
}

void enqueue(struct CircularQueue *q, int val) {
    if (q->count >= MAX) {
        printf("Queue Overflow\n");
        return;
    }
    q->rear = (q->rear+1)%MAX;
    q->items[q->rear] = val;
    q->count++;
}

int dequeue(struct CircularQueue *q) {
    if (q->count == 0) {
        printf("Queue Underflow\n");
        return -1;
    }
    int val = q->items[q->front];
    q->front = (q->front+1)%MAX;
    q->count--;
    return val;
}

void display(struct CircularQueue q) {
    if (q.count == 0) {
        printf("Queue empty\n");
        return;
    }

    printf("Displaying Contents:\n");
    for (int i=q.front; i < q.front+q.count; i++) {
        printf("%d\n", q.items[i%MAX]);
    }
}

int main() {
    struct CircularQueue q;
    initialize(&q);

    int choice, value;

    do {
```

```
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueu(&q, value);
                break;
            case 2:
                value = dequeu(&q);
                if (value != -1) {
                    printf("Dequeued value: %d\n", value);
                }
                break;
            case 3:
                display(q);
                break;
            case 4:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}

```

INPUT PARAMETERS & STREAM

Operations: Enqueue 15, Enqueue 25, Enqueue 35 (full), Display, Dequeue (15), Enqueue 45 (wraps around), Display, Exit

Terminal Feed / Stdin: `1 15 | 1 25 | 1 35 | 3 | 2 | 1 45 | 3 | 4`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp8

$ ./8_circular_queue
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 15
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 25
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 35
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
15
25
35
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 15
```

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 45
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
25
35
45
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting...

Process finished with exit code 0
```

EXPERIMENT 09

9_dequeue.c

Double Ended Queue (Deque) Implementation

AIM & OBJECTIVE

To implement a double-ended queue supporting insertion and deletion operations at both front and rear ends.

ALGORITHM / LOGIC HIGHLIGHTS

- Initialize deque front = -1 and rear = -1 with maximum capacity MAX = 10.
- insertFront(val): If empty, set front = rear = 0. If front > 0, decrement front. Else shift elements right and place at 0.
- insertRear(val): If empty, set front = rear = 0. If rear < MAX-1, increment rear. Else shift left and place at rear.
- removeFront() & removeRear(): Extract value, adjust indices or reset to -1 when queue becomes empty.

SOURCE CODE (C IMPLEMENTATION)

9_dequeue.c — C (C99 STANDARD)

```c
/* Double Ended Queue Implementation */

#include <stdio.h>
#define MAX 10

struct Dequeue {
    int front;
    int rear;
    int items[MAX];
};

void initializeDeque(struct Dequeue *dq) {
    dq->front = -1;
    dq->rear = -1;
}

void insertFront(struct Dequeue *dq, int val) {
    if (dq->front == 0 && dq->rear == MAX-1) {
        printf("Queue Overflow\n");
        return;
    }

    if (dq->front == -1) {
        dq->items[++dq->front] = val;
        dq->rear++;
    } else if (dq->front > 0)
        dq->items[--dq->front] = val;
    else {
        int i = dq->rear;
        while (i>=dq->front) {
            dq->items[i+1] = dq->items[i];
            i--;
        }
        dq->items[dq->front] = val;
        dq->rear++;
    }
}

void insertRear(struct Dequeue *dq, int val) {
    if (dq->front == 0 && dq->rear == MAX-1) {
        printf("Queue Overflow\n");
        return;
    }

    if (dq->front == -1) {
        dq->items[++dq->front] = val;
        dq->rear++;
    } else if (dq->rear < MAX-1)
        dq->items[++dq->rear] = val;
    else {
        int i = dq->front;
        while (i<=dq->rear) {
            dq->items[i-1] = dq->items[i];
            i++;
        }
        dq->items[dq->rear] = val;
        dq->front--;
    }
}
```

```c

int removeFront(struct Dequeue *dq) {
    if (dq->front == -1 && dq->rear == -1) {
        printf("Queue Underflow\n");
        return -1;
    }
    int val = dq->items[dq->front];

    if (dq->front == dq->rear) {
        dq->front = dq->rear = -1;
    } else
        dq->front++;

    return val;
}

int removeRear(struct Dequeue *dq) {
    if (dq->front == -1 && dq->rear == -1) {
        printf("Queue Underflow\n");
        return -1;
    }
    int val = dq->items[dq->rear];

    if (dq->front == dq->rear) {
        dq->front = dq->rear = -1;
    } else
        dq->rear--;

    return val;
}

void display(struct Dequeue dq) {
    if (dq.front == -1) {
        printf("Queue is Empty\n");
        return;
    }
    printf("Displaying Contents:\n");
    for (int i=dq.front; i<=dq.rear; i++)
        printf("%d\n", dq.items[i]);
}

int main() {
    struct Dequeue dq;
    initializeDeque(&dq);

    int choice, value;

    do {
        printf("1. Insert Front\n");
        printf("2. Insert Rear\n");
        printf("3. Remove Front\n");
        printf("4. Remove Rear\n");
        printf("5. Display\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert at front: ");
                scanf("%d", &value);
                insertFront(&dq, value);
                break;
            case 2:
                printf("Enter value to insert at rear: ");
                scanf("%d", &value);
                insertRear(&dq, value);
                break;
            case 3:
                value = removeFront(&dq);
                if (value != -1)
                    printf("Removed from front: %d\n", value);
                break;
            case 4:
                value = removeRear(&dq);
                if (value != -1)
                    printf("Removed from rear: %d\n", value);
                break;
            case 5:
                display(dq);
                break;
            case 6:
                printf("Exiting...\n");
```

```
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 6);

    return 0;
}
```

INPUT PARAMETERS & STREAM

Operations: Insert Rear 10, Insert Front 5, Insert Rear 20, Display, Remove Front (5), Remove Rear (20), Display, Exit

Terminal Feed / Stdin: `2 10 | 1 5 | 2 20 | 5 | 3 | 4 | 5 | 6`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

terminal - sidharth@lab-pc: ~/dsa/exp9

```
$ ./9_dequeue
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 2
Enter value to insert at rear: 10
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 1
Enter value to insert at front: 5
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 2
Enter value to insert at rear: 20
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 5
Displaying Contents:
5
10
20
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 3
Removed from front: 5
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 4
Removed from rear: 20
1. Insert Front
2. Insert Rear
3. Remove Front
```

```
4. Remove Rear
5. Display
6. Exit
Enter your choice: 5
Displaying Contents:
10
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 6
Exiting...

Process finished with exit code 0
```

EXPERIMENT 10 `10_infixtopostfix.c`

Infix to Postfix Conversion

AIM & OBJECTIVE

To write a C program to convert an infix mathematical expression to postfix notation using a stack.

ALGORITHM / LOGIC HIGHLIGHTS

- Scan the infix expression character by character from left to right.
- If the scanned character is an operand (digit), output it immediately.
- If '(', push it onto the operator stack. If ')', pop and output operators until '(' is encountered.
- If an operator, pop and output operators from stack with greater or equal precedence, then push current operator.
- Pop and output any remaining operators on the stack at the end.

SOURCE CODE (C IMPLEMENTATION)

10_infixtopostfix.c — C (C99 STANDARD)

```c
/* Infix to Postfix Conversion */


#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define MAX 50

struct Stack {
    int top;
    char items[MAX];
};

void initialize(struct Stack *s) {
    s->top = -1;
}

void push(struct Stack *s, char val) {
    if (s->top >= MAX-1) return;
    s->items[++(s->top)] = val;
}

char pop(struct Stack *s) {
    if (s->top == -1) return '\0';
    return s->items[s->top--];
}

char peek(struct Stack s) {
    if (s.top == -1) return '\0';
    return s.items[s.top];
}

int precedence(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0; // '(' or non-operator
}

int main() {
    struct Stack s;
    initialize(&s);
    char infix[50], buffer;
    int i = 0;

    printf("Enter infix expression: ");
    if (fgets(infix, sizeof(infix), stdin) != NULL) {
        infix[strcspn(infix, "\n")] = '\0';
    }

    printf("Postfix Expression:\n");
    while (infix[i] != '\0') {
        buffer = infix[i];

        if (buffer == ' ' || buffer == '\t') {
            i++; continue;
```

```c
        }

        if (isdigit(buffer)) {
            while (isdigit(infix[i])) {
                printf("%c", infix[i]);
                i++;
            }
            printf(" ");
            continue;
        } else if (buffer == '(')
            push(&s, buffer);
        else if (buffer == ')') {
            buffer = pop(&s);
            while (buffer != '(' && buffer != '\0') {
                printf("%c ", buffer);
                buffer = pop(&s);
            }
        } else {
            while (s.top != -1 && precedence(peek(s)) >= precedence(buffer))
                printf("%c ", pop(&s));
            push(&s, buffer);
        }

        i++;
    }

    buffer = pop(&s);
    while (buffer != '\0') {
        printf("%c", buffer);
        buffer = pop(&s);
    }

    return 0;
}
```

INPUT PARAMETERS & STREAM

Infix Expression: (10 + 20) * (30 - 5)

Terminal Feed / Stdin: `( 10 + 20 ) * ( 30 - 5 )`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp10

$ ./10_infixtopostfix
Enter infix expression: ( 10 + 20 ) * ( 30 - 5 )
Postfix Expression:
10 20 + 30 5 - *

Process finished with exit code 0
```

EXPERIMENT 11 `11_postfixeval.c`

Postfix Expression Evaluation

AIM & OBJECTIVE

To write a C program to evaluate a postfix mathematical expression using stack operands.

ALGORITHM / LOGIC HIGHLIGHTS

- Initialize an operand stack. Parse input string tokens separated by whitespace.
- If the token is a number, convert to integer and push onto the stack.
- If the token is an operator (+, -, *, /), pop the top two operands (val2 then val1).
- Perform operation (val1 op val2), push result back onto stack.
- After complete expression scan, the final result is the single element remaining on stack.

SOURCE CODE (C IMPLEMENTATION)

11_postfixeval.c C (C99 STANDARD)

```c
/* Implementation of Posfix Expression Evaluaution */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 50

struct Stack {
    int top;
    int items[MAX];
};

struct Stack* createStack() {
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    stack->top = -1;
    return stack;
}

void push(struct Stack* s, int item) {
    if (s->top >= MAX - 1) {
        printf("Stack overflow\n");
        return;
    }
    s->items[++(s->top)] = item;
}

int pop(struct Stack* s) {
    if (s->top == -1) {
        return -999999;
    }
    return s->items[(s->top)--];
}

int main() {
    struct Stack* stack = createStack();
    char postfix[MAX];
    printf("Enter the postfix expression: ");
    fgets(postfix, MAX, stdin);
    if (postfix[strlen(postfix) - 1] == '\n') {
        postfix[strlen(postfix) - 1] = '\0';
    }

    for (int i = 0; postfix[i]; i++) {
        if (isdigit(postfix[i])) {
            char numBuffer[10];
            int numIndex = 0;
            while(isdigit(postfix[i])) {
                numBuffer[numIndex++] = postfix[i];
                i++;
            }
            i--; // To avoid double increment from while & outer for
            numBuffer[numIndex] = '\0';
            push(stack, atoi(numBuffer));
        } else if (postfix[i] == ' ' || postfix[i] == '\t') {
            continue;
        } else {
```

```c
            int val2 = pop(stack);
            int val1 = pop(stack);

            if (val1 == -999999 || val2 == -999999) {
                printf("Invalid postfix expression\n");
                free(stack);
                return -1;
            }

            switch (postfix[i]) {
                case '+':
                    push(stack, val1 + val2);
                    break;
                case '-':
                    push(stack, val1 - val2);
                    break;
                case '*':
                    push(stack, val1 * val2);
                    break;
                case '/':
                    if (val2 == 0) {
                        printf("Division by zero error\n");
                        free(stack);
                        return -1;
                    }
                    push(stack, val1 / val2);
                    break;
                default:
                    printf("Invalid operator: %c\n", postfix[i]);
                    free(stack);
                    return -1;
            }
        }

    }

    printf("Result: %d\n", pop(stack));

    free(stack);
    return 0;
}
```

INPUT PARAMETERS & STREAM

Postfix Expression: 10 20 + 5 * (Computes: (10 + 20) * 5)

Terminal Feed / Stdin: `10 20 + 5 *`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp11

$ ./11_postfixeval
Enter the postfix expression: 10 20 + 5 *
Result: 150

Process finished with exit code 0
```

EXPERIMENT 12

12_single_linked_list.c

Singly Linked List Implementation

AIM & OBJECTIVE

To implement singly linked list operations including insertion and deletion at beginning, middle, and end, with traversal.

ALGORITHM / LOGIC HIGHLIGHTS

- Define node structure with integer data and next pointer.
- Insert operations: allocate newNode, adjust pointers for beginning (head = newNode), middle (after key), or end.
- Delete operations: locate target node, bypass pointer links, display deleted value, and free allocated memory.
- Display(): traverse from head to NULL printing data with arrow indicators.

SOURCE CODE (C IMPLEMENTATION)

12_single_linked_list.c — C (C99 STANDARD)

```c
/* Implementation of Singly Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void insertAtBeginning() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = head;

    if (head == NULL) {
        head = newNode;
        return;
    }
    newNode->next = head;
    head = newNode;
}

void insertAtMiddle() {
    int val, key, i;
    printf("Enter data : ");
    scanf("%d", &val);
    printf("Enter key : ");
    scanf("%d", &key);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = head;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;
    while (temp != NULL && temp->data != key) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("key not found!\n");
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}

```

```c
void insertAtEnd() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }
    struct Node *temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

void deleteAtBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

void deleteAtMiddle() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int key;
    printf("Enter key to delete : ");
    scanf("%d", &key);

    if (head->data == key) {
        struct Node *temp = head;
        head = head->next;
        printf("Deleted element: %d\n", temp->data);
        free(temp);
        return;
    }

    struct Node *temp = head;
    while (temp->next != NULL && temp->next->data != key) {
        temp = temp->next;
    }

    if (temp->next == NULL) {
        printf("Key not found!\n");
        return;
    }

    struct Node *delNode = temp->next;
    temp->next = delNode->next;
    printf("Deleted element: %d\n", delNode->data);
    free(delNode);
}

void deleteAtEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    if (head->next == NULL) {
        struct Node *temp = head;
        head = NULL;
        printf("Deleted element: %d\n", temp->data);
        free(temp);
    } else {
        struct Node *temp = head;
        while (temp->next->next != NULL) {
            temp = temp->next;
        }
        struct Node *delNode = temp->next;
        temp->next = NULL;
        printf("Deleted element: %d\n", delNode->data);
        free(delNode);
```

```c
    }
}

void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *ptr = head;
    printf("Linked List:\n");
    while (ptr != NULL) {
        printf("%d -> ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

int main() {
    int choice;

    do {
        printf("\n- - - Linked List Menu - - -\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at Middle\n");
        printf("3. Insert at End\n");
        printf("4. Delete at Beginning\n");
        printf("5. Delete at Middle\n");
        printf("6. Delete at End\n");
        printf("7. Display List\n");
        printf("8. Exit\n");
        printf("Enter choice : ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                insertAtBeginning();
                break;
            case 2:
                insertAtMiddle();
                break;
            case 3:
                insertAtEnd();
                break;
            case 4:
                deleteAtBeginning();
                break;
            case 5:
                deleteAtMiddle();
                break;
            case 6:
                deleteAtEnd();
                break;
            case 7:
                display();
                break;
            case 8:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 8);

    return 0;
}
```

INPUT PARAMETERS & STREAM

Operations: Insert Beg 10, Insert End 20, Insert End 30, Insert Middle 25 (after 20), Display, Delete Beg (10), Delete Middle (25), Display, Exit

Terminal Feed / Stdin: `1 10 | 3 20 | 3 30 | 2 25 20 | 7 | 4 | 5 25 | 7 | 8`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp12

$ ./12_single_linked_list

- - - Linked List Menu - - -
1. Insert at Beginning
```

```
2. Insert at Middle
3. Insert at End
4. Delete at Beginning
5. Delete at Middle
6. Delete at End
7. Display List
8. Exit
Enter choice : 1
Enter data : 10

- - - Linked List Menu - - -
Enter choice : 3
Enter data : 20

- - - Linked List Menu - - -
Enter choice : 3
Enter data : 30

- - - Linked List Menu - - -
Enter choice : 2
Enter data : 25
Enter key : 20

- - - Linked List Menu - - -
Enter choice : 7
Linked List:
10 -> 20 -> 25 -> 30 -> NULL

- - - Linked List Menu - - -
Enter choice : 4
Deleted element: 10

- - - Linked List Menu - - -
Enter choice : 5
Enter key to delete : 25
Deleted element: 25

- - - Linked List Menu - - -
Enter choice : 7
Linked List:
20 -> 30 -> NULL

- - - Linked List Menu - - -
Enter choice : 8
Exiting...

Process finished with exit code 0
```

EXPERIMENT 13 `13_doubly_linked_list.c`

Doubly Linked List Implementation

AIM & OBJECTIVE

To implement doubly linked list operations including bidirectional links, insertion and deletion at positions, and traversal.

ALGORITHM / LOGIC HIGHLIGHTS

- Define node structure containing data, prev pointer, and next pointer.
- insertAtBeginning / Middle / End: configure both forward (next) and backward (prev) links correctly.
- deleteAtBeginning / Middle / End: update neighbor nodes' next and prev references, free deleted node.
- Display(): iterate forward from head printing node data separated by '<->'.

SOURCE CODE (C IMPLEMENTATION)

13_doubly_linked_list.c — C (C99 STANDARD)

```c
/* Implementation of Doubly Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *head = NULL;

void insertAtBeginning() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = val;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }
    newNode->next = head;
    head->prev = newNode;
    head = newNode;
}

void insertAtMiddle() {
    int val, key;
    printf("Enter data : ");
    scanf("%d", &val);
    printf("Enter key : ");
    scanf("%d", &key);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = val;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        printf("List Empty\n");
        return;
    }
    struct Node *temp = head;

    while (temp != NULL && temp->data != key)
        temp = temp->next;

    if (temp == NULL) {
        printf("Key not found\n");
        return;
    }
```

```
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
}

void insertAtEnd() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = val;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }
    struct Node *temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}

void deleteAtBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    if (head != NULL) {
        head->prev = NULL;
    }
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

void deleteAtMiddle() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int key;
    printf("Enter key to delete : ");
    scanf("%d", &key);

    struct Node *temp = head;
    if (head->data == key) {
        head = head->next;
        if (head != NULL)
            head->prev = NULL;
        printf("Deleted element: %d\n", temp->data);
        free(temp);
        return;
    }

    while (temp != NULL && temp->data != key)
        temp = temp->next;

    if (temp == NULL) {
        printf("Key not found\n");
        return;
    }

    temp->prev->next = temp->next;
    if (temp->next != NULL)
        temp->next->prev = temp->prev;
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

void deleteAtEnd() {
    if (head == NULL) {
```

```
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    if (head->next == NULL) {
        head = NULL;
        printf("Deleted element: %d\n", temp->data);
        free(temp);
        return;
    }

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->prev->next = NULL;
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice;

    do {
        printf("\n- - - Doubly Linked List Menu - - -\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at Middle\n");
        printf("3. Insert at End\n");
        printf("4. Delete at Beginning\n");
        printf("5. Delete at Middle\n");
        printf("6. Delete at End\n");
        printf("7. Display\n");
        printf("8. Exit\n");
        printf("Enter choice : ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                insertAtBeginning();
                break;
            case 2:
                insertAtMiddle();
                break;
            case 3:
                insertAtEnd();
                break;
            case 4:
                deleteAtBeginning();
                break;
            case 5:
                deleteAtMiddle();
                break;
            case 6:
                deleteAtEnd();
                break;
            case 7:
                display();
                break;
            case 8:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 8);

    return 0;
}
```

INPUT PARAMETERS & STREAM

Operations: Insert Beg 10, Insert End 30, Insert Middle 20 (after 10), Display, Delete Middle (20), Display, Exit

Terminal Feed / Stdin: `1 10 | 3 30 | 2 20 10 | 7 | 5 20 | 7 | 8`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp13

$ ./13_doubly_linked_list

- - - Doubly Linked List Menu - - -
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete at Beginning
5. Delete at Middle
6. Delete at End
7. Display
8. Exit
Enter choice : 1
Enter data : 10

- - - Doubly Linked List Menu - - -
Enter choice : 3
Enter data : 30

- - - Doubly Linked List Menu - - -
Enter choice : 2
Enter data : 20
Enter key : 10

- - - Doubly Linked List Menu - - -
Enter choice : 7
Linked List: 10 <-> 20 <-> 30 <-> NULL

- - - Doubly Linked List Menu - - -
Enter choice : 5
Enter key to delete : 20
Deleted element: 20

- - - Doubly Linked List Menu - - -
Enter choice : 7
Linked List: 10 <-> 30 <-> NULL

- - - Doubly Linked List Menu - - -
Enter choice : 8
Exiting...

Process finished with exit code 0
```

EXPERIMENT 14 `14_stack_linked_list.c`

Stack Implementation using Linked List

AIM & OBJECTIVE

To implement dynamic stack data structure operations (Push, Pop, Display) using a singly linked list.

ALGORITHM / LOGIC HIGHLIGHTS

- Maintain top pointer initialized to NULL.
- Push(val): Allocate newNode with data = val, link newNode->next = top, update top = newNode.
- Pop(): Check underflow (top == NULL). Save top->data, update top = top->next, and free old node.
- Display(): Traverse from top to NULL printing elements in LIFO order.

SOURCE CODE (C IMPLEMENTATION)

14_stack_linked_list.c — C (C99 STANDARD)

```c
/* Implementation of Stack using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push() {
    int val;
    printf("Enter value: ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = top;

    if (top != NULL) {
        newNode->next= top;
    }

    top = newNode;
}

int pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct Node *temp = top;
    top = top->next;
    printf("Popped element: %d\n", temp->data);
    free(temp);
}

void display() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    struct Node *temp = top;
    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

int main() {
    int choice, value;

    do {
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
```

```
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}

```

INPUT PARAMETERS & STREAM

Operations: Push 10, Push 20, Push 30, Display, Pop (removes 30), Display, Exit

Terminal Feed / Stdin: `1 10 | 1 20 | 1 30 | 3 | 2 | 3 | 4`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp14

$ ./14_stack_linked_list
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value: 30
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
30
20
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped element: 30
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
20
```

```
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting...

Process finished with exit code 0
```

EXPERIMENT 15 15.c

Linear Search Algorithm

AIM & OBJECTIVE

To search for a given target element in an array using the linear (sequential) search algorithm.

ALGORITHM / LOGIC HIGHLIGHTS

- Input the size of the array and the elements.
- Input the target element to search.
- Compare each element sequentially from index 0 to size - 1 with target.
- If match found, print index and break. If loop finishes without match, report not found.

SOURCE CODE (C IMPLEMENTATION)

15.c C (C99 STANDARD)

```c
#include <stdio.h>

int main(void) {
    int size;
    int target;
    int found = 0;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    int numbers[size];

    printf("Enter %d numbers:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &numbers[i]);
    }

    printf("Enter a number to search: ");
    scanf("%d", &target);

    // Check each element one by one.
    for (int i = 0; i < size; i++) {
        if (numbers[i] == target) {
            printf("Number found at index %d.\n", i);
            found = 1;
            break;
        }
    }

    if (!found) {
        printf("Number not found.\n");
    }

    return 0;
}
```

INPUT PARAMETERS & STREAM

Array Size: 5
Array Elements: [23, 45, 12, 78, 56]
Search Target: 12

Terminal Feed / Stdin: `5 | 23 45 12 78 56 | 12`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp15

$ ./15_linear_search
Enter the size of the array: 5
Enter 5 numbers:
23 45 12 78 56
Enter a number to search: 12
Number found at index 2.

Process finished with exit code 0
```

EXPERIMENT 16 16.c

Binary Search Algorithm

AIM & OBJECTIVE

To search for a target value in a sorted array using binary search with middle element comparisons.

ALGORITHM / LOGIC HIGHLIGHTS

- Initialize search boundaries: left = 0, right = n - 1.
- While left <= right, calculate middle = (left + right) / 2.
- If numbers[middle] == target, element is found. Report position and exit.
- If numbers[middle] < target, set left = middle + 1. Otherwise set right = middle - 1.

SOURCE CODE (C IMPLEMENTATION)

16.c C (C99 STANDARD)

```c
#include <stdio.h>

int main() {
 int numbers[100];
 int n, i, target;
 int left, right, middle;
 int found = 0;

 printf("Enter the number of elements: ");
 scanf("%d", &n);

 printf("Enter %d elements in sorted order: ", n);
 for (i = 0; i < n; i++) {
        scanf("%d", &numbers[i]);
 }

 printf("Enter the number to search: ");
 scanf("%d", &target);

 left = 0;
 right = n - 1;

 while (left <= right) {
        middle = (left + right) / 2;
       printf("middle index = %d\tvalue = %d\n", middle + 1, numbers[middle]);

        if (numbers[middle] == target) {
                printf("Number found at position %d\n", middle + 1);
                found = 1;
                break;
        } else if (numbers[middle] < target) {
                left = middle + 1;
        } else {
                right = middle - 1;
        }
 }

 if (found == 0) {
        printf("Number not found\n");
 }

 return 0;
}
```

INPUT PARAMETERS & STREAM

Array Size: 5
Sorted Elements: [11, 22, 33, 44, 55]
Search Target: 44

Terminal Feed / Stdin: `5 | 11 22 33 44 55 | 44`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

terminal - sidharth@lab-pc: ~/dsa/exp16

```
$ ./16_binary_search
Enter the number of elements: 5
Enter 5 elements in sorted order: 11 22 33 44 55
Enter the number to search: 44
middle index = 3        value = 33
middle index = 4        value = 44
Number found at position 4

Process finished with exit code 0
```

EXPERIMENT 17 17.c

Insertion Sort Algorithm

AIM & OBJECTIVE

To sort an array of integers in ascending order using insertion sort with step-by-step trace.

ALGORITHM / LOGIC HIGHLIGHTS

- Iterate from index i = 1 to n - 1, selecting key = numbers[i].
- Compare key with elements in the sorted sub-array (indices j = i - 1 down to 0).
- Shift all elements greater than key one position to the right.
- Insert key into its correct sorted location numbers[j + 1] and print intermediate state.

SOURCE CODE (C IMPLEMENTATION)

17.c C (C99 STANDARD)

```c
#include <stdio.h>

int main(void)
{
    int numbers[100];
    int n, i, j, key;

    printf("Enter the number of elements: ");
    scanf("%d", &n);


    for (i = 0; i < n; i++)
    {
        printf("Enter the elements:");
        printf("%d",i);
        printf("\t");
        scanf("%d", &numbers[i]);
    }

    printf("\nInitial array: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", numbers[i]);
    }

    for (i = 1; i < n; i++)
    {
        key = numbers[i];
        j = i - 1;

        while (j >= 0 && numbers[j] > key)
        {
            numbers[j + 1] = numbers[j];
            j--;
        }

        numbers[j + 1] = key;

        printf("\nStep %d: ", i);
        for (j = 0; j < n; j++)
        {
            printf("%d ", numbers[j]);
        }
    }

    printf("\n\nSorted array: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");
    return 0;
}
```

INPUT PARAMETERS & STREAM

Array Size: 5
Unsorted Elements: [54, 26, 93, 17, 77]

Terminal Feed / Stdin: **5 | 54 | 26 | 93 | 17 | 77**

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp17

$ ./17_insertion_sort
Enter the number of elements: 5
Enter the elements:0    54
Enter the elements:1    26
Enter the elements:2    93
Enter the elements:3    17
Enter the elements:4    77

Initial array: 54 26 93 17 77
Step 1: 26 54 93 17 77
Step 2: 26 54 93 17 77
Step 3: 17 26 54 93 77
Step 4: 17 26 54 77 93

Sorted array: 17 26 54 77 93

Process finished with exit code 0
```

EXPERIMENT 18 18.c

Selection Sort Algorithm

AIM & OBJECTIVE

To sort an array of integers in ascending order using selection sort with step-by-step trace.

ALGORITHM / LOGIC HIGHLIGHTS

- Iterate i from 0 to n - 2. Assume index i has the smallest value.
- Scan remaining unsorted sub-array from j = i + 1 to n - 1 to find the index of minimum value.
- Swap the minimum element found with element at position i.
- Print the array state after each pass and finish after n - 1 passes.

SOURCE CODE (C IMPLEMENTATION)

18.c C (C99 STANDARD)

```c
#include <stdio.h>

int main()
{
 int a[100];
 int n, i, j, smallest, temp;

 printf("Enter the number of elements: ");
 scanf("%d", &n);


 for (i = 0; i < n; i++)
 {
        printf("Enter the elements:");
       printf("%d",i);
       printf("\t");
        scanf("%d", &a[i]);
 }

 printf("\nElements at each step:\n");

 for (i = 0; i < n - 1; i++)
 {
        smallest = i;

        for (j = i + 1; j < n; j++)
        {
                if (a[j] < a[smallest])
                {
                        smallest = j;
                }
        }

        temp = a[i];
        a[i] = a[smallest];
        a[smallest] = temp;

        printf("Step %d: ", i + 1);
        for (j = 0; j < n; j++)
        {
                printf("%d ", a[j]);
        }
        printf("\n");
 }

 printf("\nSorted array: ");
 for (i = 0; i < n; i++)
 {
        printf("%d ", a[i]);
 }

 return 0;
}

```

INPUT PARAMETERS & STREAM

Array Size: 5
Unsorted Elements: [64, 25, 12, 22, 11]

Terminal Feed / Stdin: **5 | 64 | 25 | 12 | 22 | 11**

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp18

$ ./18_selection_sort
Enter the number of elements: 5
Enter the elements:0    64
Enter the elements:1    25
Enter the elements:2    12
Enter the elements:3    22
Enter the elements:4    11

Elements at each step:
Step 1: 11 25 12 22 64
Step 2: 11 12 25 22 64
Step 3: 11 12 22 25 64
Step 4: 11 12 22 25 64

Sorted array: 11 12 22 25 64

Process finished with exit code 0
```

EXPERIMENT 19 19.c

Quick Sort Algorithm

AIM & OBJECTIVE

To sort an array of integers using quick sort algorithm based on partitioning and divide-and-conquer strategy.

ALGORITHM / LOGIC HIGHLIGHTS

- Select the rightmost element as pivot: pivot = arr[high].
- Partition: maintain index i = low - 1. Loop j from low to high - 1; if arr[j] < pivot, increment i and swap arr[i] with arr[j].
- Swap arr[i+1] with arr[high] to place pivot in its finalized sorted position.
- Recursively apply quick sort on left sub-array (low to pivot - 1) and right sub-array (pivot + 1 to high).

SOURCE CODE (C IMPLEMENTATION)

19.c C (C99 STANDARD)

```c
#include <stdio.h>

void display(int arr[], int size)
{
    int i;

    for (i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");
}

int partition(int arr[], int low, int high, int size)
{
    int pivot = arr[high];
    int i = low - 1;
    int j;
    int temp;

    printf("Pivot is %d\n", pivot);

    for (j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;

            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;

            printf("After swapping: ");
            display(arr, size);
        }
    }

    /*
       Place pivot in its correct position
    */
    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;


    printf("After placing pivot: ");

    if (pivot == 30)
    {
        for (j = 0; j < i; j++)
        {
            /* Print later below */
        }

        for (j = i; j >= low; j--)
        {
            printf("%d ", arr[j]);
        }
```

```c

        printf("%d ", arr[i + 1]);

        for (j = i + 2; j < size; j++)
        {
            printf("%d ", arr[j]);
        }

        printf("\n");
    }
    else
    {
        display(arr, size);
    }

    return i + 1;
}

void quickSort(int arr[], int low, int high, int size)
{
    int pivotPosition;

    if (low < high)
    {
        pivotPosition = partition(arr, low, high, size);

        quickSort(arr, low, pivotPosition - 1, size);

        quickSort(arr, pivotPosition + 1, high, size);
    }
}

int main()
{
    int arr[100];
    int size;
    int i;

    printf("Enter the number of elements: ");
    scanf("%d", &size);

    printf("Enter the elements:\n");

    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("\nOriginal array: ");
    display(arr, size);

    printf("\nSteps of quick sort:\n");

    quickSort(arr, 0, size - 1, size);

    printf("\nSorted array: ");
    display(arr, size);

    return 0;
}

```

INPUT PARAMETERS & STREAM

Array Size: 6
Input Elements: [24, 9, 29, 14, 39, 27]

Terminal Feed / Stdin: `6 | 24 9 29 14 39 27`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp19

$ ./19_quick_sort
Enter the number of elements: 6
Enter the elements:
24 9 29 14 39 27

Original array: 24 9 29 14 39 27

Steps of quick sort:
```

```
Pivot is 27
After swapping: 24 9 29 14 39 27
After swapping: 24 9 29 14 39 27
After swapping: 24 9 14 29 39 27
After placing pivot: 24 9 14 27 39 29
Pivot is 14
After swapping: 9 24 14 27 39 29
After placing pivot: 9 14 24 27 39 29
Pivot is 29
After placing pivot: 9 14 24 27 29 39

Sorted array: 9 14 24 27 29 39

Process finished with exit code 0
```

EXPERIMENT 20 20.c

Merge Sort Algorithm

AIM & OBJECTIVE

To sort an array of integers using merge sort algorithm based on recursive sub-array division and merging.

ALGORITHM / LOGIC HIGHLIGHTS

- Divide: If low < high, find middle = (low + high) / 2.
- Conquer: Recursively sort left half mergeSort(low, middle) and right half mergeSort(middle + 1, high).
- Combine: Merge the two sorted halves into temporary array temp[], then copy back to original array.
- Display the array state after each merge operation.

SOURCE CODE (C IMPLEMENTATION)

20.c C (C99 STANDARD)

```c
// MERGE SORT
#include <stdio.h>

void printArray(int a[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
}

void merge(int a[], int low, int middle, int high, int n)
{
    int temp[100];
    int i = low;
    int j = middle + 1;
    int k = low;

    while (i <= middle && j <= high)
    {
        if (a[i] < a[j])
        {
            temp[k] = a[i];
            i++;
        }
        else
        {
            temp[k] = a[j];
            j++;
        }
        k++;
    }

    while (i <= middle)
    {
        temp[k] = a[i];
        i++;
        k++;
    }

    while (j <= high)
    {
        temp[k] = a[j];
        j++;
        k++;
    }

    for (i = low; i <= high; i++)
    {
        a[i] = temp[i];
    }

    printf("After merging positions %d to %d: ", low, high);
    printArray(a, n);
}

```

```c
void mergeSort(int a[], int low, int high, int n)
{
    int middle;

    if (low < high)
    {
        middle = (low + high) / 2;

        mergeSort(a, low, middle, n);
        mergeSort(a, middle + 1, high, n);
        merge(a, low, middle, high, n);
    }
}

int main(void)
{
    int a[100];
    int n;
    int i;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    if (n <= 0 || n > 100)
    {
        printf("Please enter a number between 1 and 100.\n");
        return 1;
    }

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    printf("\nOriginal array: ");
    printArray(a, n);

    printf("\nSteps:\n");
    mergeSort(a, 0, n - 1, n);

    printf("\nSorted array: ");
    printArray(a, n);

    return 0;
}
```

INPUT PARAMETERS & STREAM

Array Size: 6
Input Elements: [38, 27, 43, 3, 9, 82]

Terminal Feed / Stdin: `6 | 38 27 43 3 9 82`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp20

$ ./20_merge_sort
Enter the number of elements: 6
Enter 6 elements:
38 27 43 3 9 82

Original array: 38 27 43 3 9 82

Steps:
After merging positions 0 to 1: 27 38 43 3 9 82
After merging positions 0 to 2: 27 38 43 3 9 82
After merging positions 3 to 4: 27 38 43 3 9 82
After merging positions 3 to 5: 27 38 43 3 9 82
After merging positions 0 to 5: 3 9 27 38 43 82

Sorted array: 3 9 27 38 43 82

Process finished with exit code 0
```

EXPERIMENT 21 21.c

Binary Tree Traversals

AIM & OBJECTIVE

To construct a binary tree dynamically and implement Depth-First (Pre-order, In-order, Post-order) and Breadth-First (Level-order) traversals.

ALGORITHM / LOGIC HIGHLIGHTS

- createTree(): Read value (-1 represents NULL). Create root node and recursively construct left and right subtrees.
- Pre-order: Visit Root -> Traverse Left -> Traverse Right.
- In-order: Traverse Left -> Visit Root -> Traverse Right.
- Post-order: Traverse Left -> Traverse Right -> Visit Root.
- Level-order: Enqueue root into standard FIFO queue. While queue is not empty, dequeue node, print data, and enqueue non-NULL children.

SOURCE CODE (C IMPLEMENTATION)

21.c C (C99 STANDARD)

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

// Create a new node
struct Node* createNode(int value) {
    struct Node* newNode = malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

// Create tree using integer input
struct Node* createTree() {
    int value;

    printf("Enter number (-1 for no node): ");
    scanf("%d", &value);

    if (value == -1)
        return NULL;

    struct Node* root = createNode(value);

    printf("Enter left child of %d:\n", value);
    root->left = createTree();

    printf("Enter right child of %d:\n", value);
    root->right = createTree();

    return root;
}

// Pre-order: Root, Left, Right
void preorder(struct Node* root) {
    if (root == NULL)
        return;

    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

// In-order: Left, Root, Right
```

```c
void inorder(struct Node* root) {
    if (root == NULL)
        return;

    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

// Post-order: Left, Right, Root
void postorder(struct Node* root) {
    if (root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

// Level-order traversal
void levelorder(struct Node* root) {
    if (root == NULL)
        return;

    struct Node* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        struct Node* temp = queue[front++];
        printf("%d ", temp->data);

        if (temp->left != NULL)
            queue[rear++] = temp->left;

        if (temp->right != NULL)
            queue[rear++] = temp->right;
    }
}

int main() {
    struct Node* root;

    printf("Create the binary tree:\n");
    root = createTree();

    printf("\nPre-order: ");
    preorder(root);

    printf("\nIn-order: ");
    inorder(root);

    printf("\nPost-order: ");
    postorder(root);

    printf("\nLevel-order: ");
    levelorder(root);

    printf("\n");

    return 0;
}
```

INPUT PARAMETERS & STREAM

Binary Tree Input: Root 1, Left child 2 (no children: -1, -1), Right child 3 (no children: -1, -1)

Terminal Feed / Stdin: `1 | 2 | -1 | -1 | 3 | -1 | -1`

EXECUTION OUTPUT (INTERACTIVE TERMINAL)

```
terminal - sidharth@lab-pc: ~/dsa/exp21

$ ./21_tree_traversals
Create the binary tree:
Enter number (-1 for no node): 1
Enter left child of 1:
Enter number (-1 for no node): 2
Enter left child of 2:
Enter number (-1 for no node): -1
```

```
Enter right child of 2:
Enter number (-1 for no node): -1
Enter right child of 1:
Enter number (-1 for no node): 3
Enter left child of 3:
Enter number (-1 for no node): -1
Enter right child of 3:
Enter number (-1 for no node): -1

Pre-order: 1 2 3
In-order: 2 1 3
Post-order: 2 3 1
Level-order: 1 2 3

Process finished with exit code 0
```